

Kybeurre salé

Gist

On permet à l'utilisateur de collecter un large ensemble de samples, et l'aléatoire est trop "faible".
LWE contraint fortement l'espace des chiffrés ; avec suffisamment d'exemples, il peut déduire des propriétés sur la structure de l'espace sous-jacent – et dans le cas présent, la clef.

La vulnérabilité

Le serveur expose une implémentation simple et de base de Kyber/LWE. Il laisse à l'utilisateur la possibilité de demander autant de samples que souhaité. Tous les chiffrés utilisent la même clef et le même message, mais par la nature aléatoire de LWE, les résultats sont différents. Le but du challenge est de casser cette clef pour déchiffrer le flag, chiffré en AES.

L'implémentation de Kyber/LWE est standard si ce n'est pour son bruit :

```
class LGCM_PRNG:
    def __init__(self, seed):
        self.state = seed
        self.a = 16645258
        self.c = 1013904223
        self.m = 2**32

    def next_noise(self):
        self.state = (self.a * self.state + self.c) % self.m
        high_bits = (self.state >> 31)
        val = (high_bits * Q//4) + Q//10
        return val % Q
```

```
PRNG = LGCM_PRNG(time.time_ns())
```

Quelques exécutions suffisent à se convaincre qu'il ne produit que deux valeurs aléatoires ; le bruit est binaire. C'est toujours trop complexe pour un bruteforce, mais on se rapproche du problème de *Learning Parity with Noise* (LPN). On sent que le problème est plus "simple" que LWE. Dans la pratique, effectivement : avec assez de valeurs, on peut réduire le problème à la recherche de racines d'un gros polynôme. Montrons-le.

Modélisation

Notons E_1, E_2 les deux NOISE_CONSTANT, $(A_i, b_i)_i$ les samples, e_i les bruits associés, et s la clef.
Posons $Q = 3329$ et $n = 96$. Je note $(b_i)_k$ (resp. $(s)_k$) le $i^{\text{ème}}$ scalaire composant b_i (resp. s).

Pour chaque sample i , on a par définition

$$A_i \cdot s + e_i = b_i \text{ mod } Q \quad (1)$$

et car les e_i sont contraints

$$(e_i - E_1)(e_i - E_2) = 0 \text{ mod } Q \quad (2)$$

$$\Leftrightarrow e_i^2 - (E_1 + E_2)e_i + E_1E_2 = 0 \text{ mod } Q \quad (3)$$

En posant $u = -(E_1 + E_2)$ et $v = E_1E_2$, eq. 3 devient

$$(b_i - A_i \cdot s)^2 + u(b_i - A \cdot s) + v = 0 \text{ mod } Q \quad (4)$$

En notant $L = A_i \cdot s$ dans eq. 4

$$L^2 - (2b_i + u)L + ub_i + v = -b_i^2 \bmod Q \quad (5)$$

C'est un multinôme de degré 2 en inconnues $(s)_j, u, v$. C'est trop difficile à résoudre. On va le linéariser.

Linéarisation

$$L^2 = \left(\sum (A_i)_k (s)_k \right) \left(\sum (A_i)_k (s)_k \right) = \sum (A_i)_k^2 (s)_k^2 + 2 \sum_{k < p} (A_i)_k (A_i)_p (s)_k (s)_p \quad (6)$$

Et **grosse astuce** : $(s)_i \in \{0, 1\}$, ie $s_i^2 = s_i$.

$$L^2 = \sum (A_i)_k^2 (s)_k + 2 \sum_{k < p} (A_i)_k (A_i)_p (s)_k (s)_p \quad (7)$$

Pour rappel, on connaît (par les samples) les valeurs des A_i et b_i . Pour linéariser eq. 5, on va introduire les variables (inconnues) suivantes

- u, v , qu'on ne connaît pas ;
- $(s)_i u$, pour le terme linéaire en L ;
- $(s)_i$ et $(s)_i (s)_j$ pour le terme en L^2 .

Soit au total $2 + n + n + \binom{n}{2} = 4754$ inconnues.

Collecter 5000 samples devraient suffire, puis de laisser sagemath dilligemment résoudre le système.

Remarques

L'attaque tourne en temps raisonnable car on s'est restreint à du LPN ; elle reste valide pour du LWE, mais coûte beaucoup plus cher. On est hyper dépendant du nombre de valeurs possibles de l'erreur. Passer de deux valeurs possibles à cinq valeurs de bruit possibles (par exemple $\{-2, \dots, 2\}$) pousse le nombre de variables à $\binom{n}{5} = 6.1 * 10^7$. Bonne chance pour manipuler votre matrice carrée à 61 millions d'entrées.

Contrairement à BKZ/LLL, la dimension n de l'espace ne nous impacte que peu (polynomial contre exponentiel). Normalement, vous n'avez pas pu attaquer le problème avec du LLL.

Enfin, on a passé sous silence

1. Que le système admet bien une solution
2. Et qu'elle est unique.

C'est évident, mais c'est en fait le gros du travail. Voir [1] pour la preuve (et l'idée générale du challenge).

Bibliography

- [1] S. Arora and R. Ge, "New algorithms for learning in presence of errors," in *International Colloquium on Automata, Languages, and Programming*, 2011, pp. 403–415.